40 Days of LeetCode Challenge

40 days, 40 problems
DAY :10

LEETCODE PROBLEM NO. **1480 : Running sum of 1D array**

Problem Link : https://leetcode.com/problems/running-sum-of-1d-array/

Easy · 1D Arrays · Nested Loops · Prefix Sum · In-place operations · Cumulative addition.

STEP BY STEP SOLUTION

Step 1. Understand the question :-

We're given an integer array nums.

We need to return a new array where each position contains the sum of all elements from the beginning of the origi nal array up to that position.

Mathematically: runningSum[i] = nums[0] + nums[1] +……..+ nums[i]

In simple terms, every element in the resulting array represents the cumulative sum up to that point.

Step 2. Translating it into simpler language with examples :-

Example 1:
Input: nums = [1,2,3,4]
Output: [1,3,6,10]
Explanation: Running sum is obtained as follows: [1, 1+2, 1+2+3, 1+2+3+4].

We keep adding the next number to the previous total.

Position	Calculation	Running sum
0	1	1
1	1 + 2	3
2	1 + 2 + 3	6
3	1 + 2 + 3 + 4	10

Therefore:

ORIGINAL ARRAY:-

1 **2** **3** **4**

⬇

RUNNING SUM:-

1 **3** **6** **10**

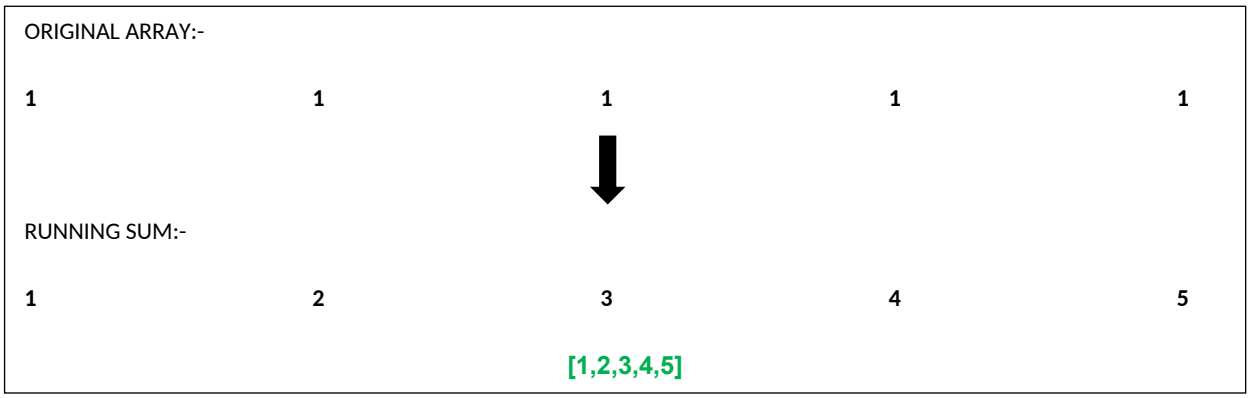
[1, 3, 6, 10]

Example 2:
Input: nums = [1,1,1,1,1]
Output: [1,2,3,4,5]
Explanation: Running sum is obtained as follows: [1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1].

We keep adding the next number to the previous total.

Position	Calculation	Running sum
0	1	1
1	1 + 1	2
2	1 + 1 + 1	3
3	1 + 1 + 1 + 1	4
4	1 + 1 + 1 + 1 +1	5

Therefore:

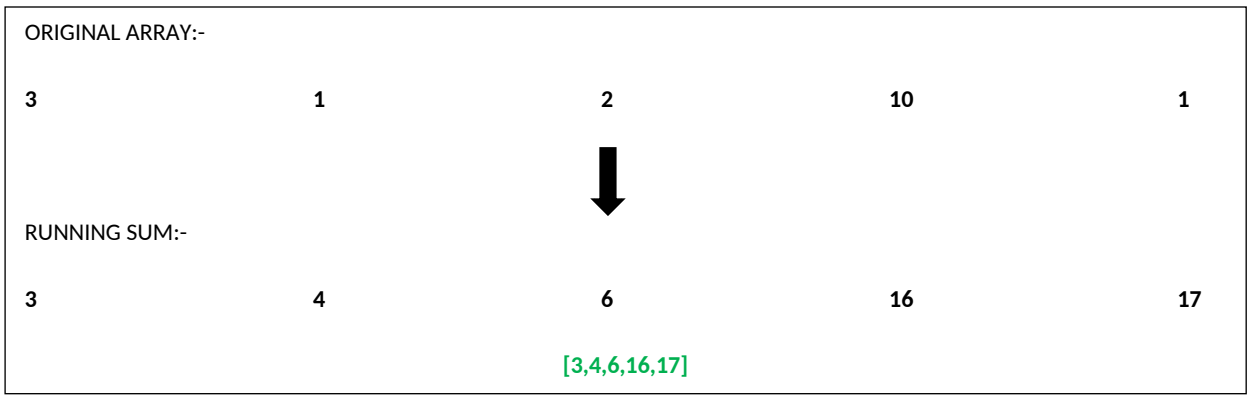


Example 3:
Input: nums = [3,1,2,10,1]
Output: [3,4,6,16,17]
Explanation: Running sum is obtained as follows: [3, 3+1, 3+1+2, 3+1+2+10, 3+1+2+10+1].

We keep adding the next number to the previous total.

Position	Calculation	Running sum
0	3	3
1	3 + 1	4
2	3 + 1 + 2	6
3	3 + 1 + 2 + 10	16
4	3 + 1 + 2 + 10 + 1	17

Therefore:



Step 3. Identify the pattern

The concept is Prefix Sum.A prefix sum stores the cumulative total of array elements from index 0 up to the current index.

The key observation: runningSum[i] = nums[0] + nums[1] +……..+ nums[i]

For the first element, the running sum is simply nums[0].

For every subsequent element, we add the current number to the previous running sum.

Step 4. Build the logic

Here's the approach we'll follow. We'll use the previous element's running sum to calculate the next one.

Algorithm

1. Start from index 1, because the first element is already its own running sum.
2. Add the previous element's running sum to the current element.
3. Store the result in the current position.
4. Repeat until the end of the array.
5. Return the updated array.

Pseudocode

```
START

  FOR i = 1 TO nums.length - 1

    nums[i] = nums[i] + nums[i - 1]

  RETURN nums

END
```

Step 5. Code in Java

This is the standard arithmetic solution for LeetCode 1480.

```
class Solution {

  public int[] runningSum(int[] nums) {

    for (int i = 1; i < nums.length; i++) {

      nums[i] += nums[i - 1];

    } return nums;

  }

}
```

How to submit

1. Open LeetCode Problem 1480. Running Sum of 1d Array
2. Select Java as the programming language.
3. Enter the method in the provided Solution class, replacing the existing method if necessary.
4. Click Run to test the examples.
5. Click Submit after the tests pass.

Step 6. Explanation & dry-run

Explanation of the code :-

A. *class Solution {*

Defines the class required by LeetCode.

B. *public int[ ] runningSum(int[ ] nums) {*

Defines a method that accepts an integer array and returns an integer array.

NOTE:-

int[ ] is the return type because we're returning an array, not a single integer.

C. *for (int i = 1; i < nums.length; i++) {*

Starts a loop from index 1.

NOTE:-

Why not index 0?

Because the first element doesn't need any calculation. Its running sum is already its original value.

D. *nums[i] += nums[i - 1];*

It adds the previous running sum to the current element and stores the result in the current position.

It's equivalent to: nums[i] = nums[i] + nums[i - 1];

For example:

If nums = [1, 2, 3, 4]:

- At i = 1: nums[1] = 2 + 1 = 3
- At i = 2: nums[2] = 3 + 3 = 6
- At i = 3: nums[3] = 4 + 6 = 10

Notice how nums[i - 1] already contains the previous running sum

E. *return nums;*

Returns the updated array containing all the running sums.

Complete dry run

Input: nums = [3, 1, 2, 10, 1]

Initially, the array is: [3, 1, 2, 10, 1]

Iteration	Index i	Calculation	Updated array
Initial	—	—	[3, 1, 2, 10, 1]
1	1	1 + 3 = 4	[3, 4, 2, 10, 1]
2	2	2 + 4 = 6	[3, 4, 6, 10, 1]
3	3	10 + 6 = 16	[3, 4, 6, 16, 1]
4	4	1 + 16 = 17	[3, 4, 6, 16, 17]

FINAL OUTPUT :- [3, 4, 6, 16, 17]

Step 7. Time complexity & Auxiliary space complexity

Time complexity: (O(n))

We traverse the array once, starting at index 1. Each iteration performs one addition.

Auxiliary space complexity: (O(1))

We update the original array directly and use only the loop variable i. No additional array is created.

NOTE:-

Is there any other approach?

Approach — Using a separate array

```
class Solution {
   public int[] runningSum(int[] nums) {
      int[] result = new int[nums.length];
      result[0] = nums[0];

      for (int i = 1; i < nums.length; i++) {
         result[i] = result[i - 1] + nums[i];
      } return result;
   }
}
```

Here, we create a separate array to store the running sums.

-]Time: (O(n))
- Auxiliary space: (O(n))
- The advantage is that the original input array remains unchanged.

SCREENSHOT:-


Submit
0
Premium
Code
Java
Auto
1
2 class Solution {
3 public int[] runningSum(int[] nums) {
4 for (int i = 1; i < nums.length; i++) {
5 nums[i] += nums[i - 1];
6 } return nums;
7 }
8 }
9